

Ball Machine

Madina IOI ishtirokchilarini xursand qilish uchun koptok mashinasini ixtiro qildi. Mashinaning tuzilishi quyidagicha:

- Mashina 0 dan $N - 1$ gacha indekslangan N ta **tugunlik** daraxtdan iborat. $N - 1$ tuguni **ildiz** hisoblanadi.
- har bir u ($0 \leq u < N - 1$) tuguni uchun, u tugunining **otasi** $P[u]$ ($P[u] > u$) bo'lgan tugun va u tuguni $P[u]$ ning **bola tuguni** hisoblanadi. Ildizning otasi yo'q. Bolalari bo'lmagan tugun **barg** deb ataladi.

Siz hech qaysi u tugunining otasi $P[u]$ ni, va hatto N ning o'zini ham bilmaysiz. Buning o'rniga, Madina sizga M , barglar sonini va ularning indeksleri $0, 1, \dots, M - 1$ ekanligini aytadi. Sizning vazifangiz mashinani ishga tushirish orqali uning tuzilishini aniqlashdir.

Mashinaning har bir tuguni ko'pi bilan bitta **koptok** ni sig'dira oladi va har bir koptokning **qiymati** manfiy bo'lmagan butun son. Agar tugunda x qiymatlik koptok bo'lsa, ushbu tugun x qiymatiga ega deymiz. Agar tugunda koptok bo'lsa, u **band** hisoblanadi; aks holda u **bo'sh** bo'ladi. Dastlab, har bir tugun bo'sh.

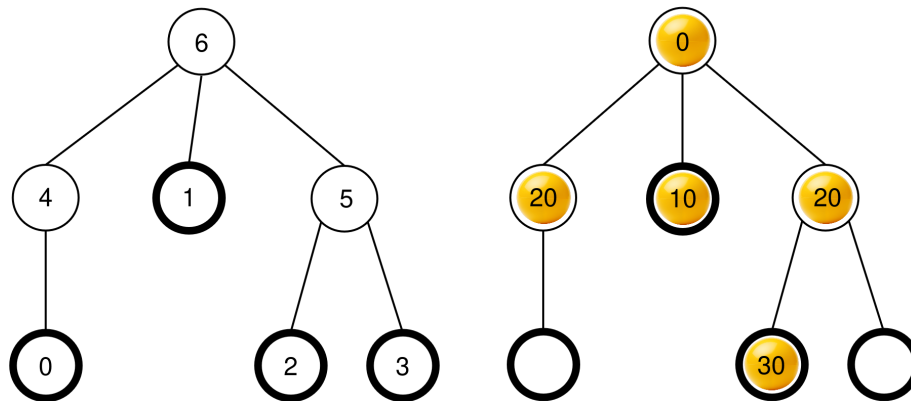
Siz ikki turdagi operatsiyalarni bajarishingiz mumkin:

- **insert** (U, X) : U bargga X qiymatlik yangi koptokni kiritishga urinish.
 - Agar U barg band bo'lsa, operatsiya koptokni kiritmasdan `false` qaytaradi.
 - Agar U barg bo'sh bo'lsa, koptok U tuguniga kiritiladi. Keyin koptok, hozir turgan tugunining otasi bo'sh bo'lsa, unga o'tadi, va bu jarayon bir necha marotaba takrorlanishi mumkin. Harakat koptok ildizga yoki otasi band bo'lgan tugunga yetganda to'xtaydi. Ushbu operatsiya `true` qaytaradi.
- **collect**() : Agar ildiz bo'sh bo'lsa (ya'ni mashina bo'sh bo'lsa), `collect` bo'sh massiv qaytaradi. Aks holda, quyida tasvirlangan `traverse` rekursiv protsedurasi mashinada hozirda mavjud bo'lgan barcha koptoklarning qiymatlarini S massivga to'playdi. Dastlab, S massivi bo'sh bo'ladi va `traverse` ($N - 1$) chaqiriladi.

```
traverse(u) :  
    u tugunidagi koptokning qiymati S oxiriga qo'shiladi  
    c[u] - u ning band bo'lgan bolalari ro'yxati bo'lsin  
    c[u] ni tugunlardagi koptoklarning qiymatlariga ko'ra kamaymaydigan  
        tartibda saralanadi (agar bir nechta tugunlardagi koptoklar bir xil qiymatga ega  
        bo'lsa, ular istalgan tartibda saralanishi mumkin)  
    c[u] dagi har bir v uchun:  
        traverse(v)
```

Ushbu protsedura tugagandan so'ng, **barcha koptoklar mashinadan chiqarib tashlanadi** va `collect` S ni qaytaradi.

Masalan, $N = 7$ ta tugun va $P = [4, 6, 5, 5, 6, 6]$ ota massivga ega mashinani ko'raylik. Chapdagi rasmda tugun indeksleri bo'lgan bo'sh mashina ko'rsatilgan, o'ngdagi rasmda esa `insert` operatsiyalari ketma-ketligidan keyin koptoklarning mumkin bo'lgan joylashuvi ko'rsatilgan (bu ketma-ketlik "Example" bo'limida keltirilgan).



`collect()` chaqirilganda, ildiz (6 tugun) band bo'ladi, shuning uchun $S = []$ ga tenglashtiriladi va `traverse(6)` chaqiriladi.

`traverse(6)` : 6-tugunning qiymati 0 ga teng; u S ga qo'shiladi va $S = [0]$ ga o'zgaradi. 6-tugunning bolalari 4, 1 va 5-tugunlar bo'lib, ularning barchasi band, qiymatlari esa mos ravishda 20, 10 va 20. Kamaymaydigan tartibda saralashdan $c[6] = [1, 5, 4]$ hosil bo'ladi. ($c[6] = [1, 4, 5]$ bo'lishi ham mumkin, chunki 4 va 5-tugunlar bir xil qiymatga ega). Keyin protsedura $c[6]$ dagi har bir tugunda shu tartibda `traverse` chaqiradi.

- **`traverse(1)`**: 1-tugunning qiymati 10 teng; u S ga qo'shiladi, $S = [0, 10]$. 1 tugunining band bo'lgan bolalari yo'q, shuning uchun bu murojaat tugaydi.
- **`traverse(5)`**: 5-tugunning qiymati 20 ga teng; S ga qo'shiladi, $S = [0, 10, 20]$. 5-tugunda bitta band bo'lgan bola, 2-tugun mavjud, shuning uchun $c[5] = [2]$ va protsedura `traverse(2)` ni chaqiradi.
 - **`traverse(2)`**: 2-tugunning qiymati 30 teng; S ga qo'shiladi, $S = [0, 10, 20, 30]$. 2-tugunning band bo'lgan bolalari yo'q, shuning uchun bu murojaat tugaydi.
 - Bajarish `traverse(5)` ga qaytadi, $c[5]$ dagi barcha bolalarga chaqiruv qilib bo'lingan. Shuning uchun 5-tugunga murojaat tugaydi.
- **`traverse(4)`** : 4-tugunning qiymati 20; S ga qo'shiladi $S = [0, 10, 20, 30, 20]$. 4-tugunning band bo'lgan bolalari yo'q, shuning uchun bu murojaat tugaydi.

Barcha chaqiruvlar yakunlandi va ko'rish uchun boshqa tugunlar yo'q. Nihoyat, har bir koptok mashinadan chiqariladi va `collect` protsedurasi $S = [0, 10, 20, 30, 20]$ massivini qaytaradi.

Sizning vazifangiz N - tugunlar sonini aniqlash va mashinaning tuzilishini tavsiflovchi, hamda daraxtning tuzilishini o'zgartirmagan holda, barg va ildiz bo'lmagan tugunlarning joylashuvi ixtiyoriy ravishda bo'lgan $R = [R[0], R[1], \dots, R[N-2]]$ ota massivini yaratish. Boshqacha qilib aytganda, yaratilgan R massivi to'g'ri hisoblanadi, agar mashinaning har bir u tuguniga $L[u]$ ($0 \leq L[u] < N$) bilan har xil labellar bilan quyidagi shartlarni qanoatlantirilgan holda tayinlash mumkin bo'lsa:

- Barcha $0 \leq u < M$ va $u = N - 1$ uchun $L[u] = u$, va
- Barcha $0 \leq u < N - 1$ uchun $L[P[u]] = R[L[u]]$.

$R[u] > u$ bo'lishi shart emas. Yechim tayyor bo'lgandan so'ng, mashina **bo'sh** bo'lishi kerak.

K - `collect` operatsiyalari soni va B - barcha `insert` chaqiruvlaridagi har qanday koptokning maksimal qiymati bo'lsin. $C = K + B$ bo'lsin. Unda $C \leq 1000$ **bo'lishi kerak**. Ba'zi subtasklardagi ballingiz C qiymatiga bog'liq.

Implementation Details

Siz quyidagi protsedurani shakllantirishingiz kerak:

```
std::vector<int> find_structure(int M)
```

- M : Mashinadagi barglar soni.
- Protsedura har bir test uchun faqatgina bir marta chaqiriladi.
- Protsedura uzunligi $N - 1$ bo'lgan $R = [R[0], R[1], \dots, R[N - 2]]$ massivini, to'g'ri ota massivini qaytarishi kerak.

Mashina bilan o'zaro aloqa uchun protsedura quyidagi ikkita protsedurani chaqirishi mumkin.

Birinchi protsedura:

```
bool insert(int U, int X)
```

- U : koptok kiritilishi kerak bo'lgan bargning indeksi. $0 \leq U < M$ sharti bajarilishi kerak.
- X : koptokdagi butun sonli qiymat. $0 \leq X \leq 1000$ sharti bajarilishi kerak.
- Agar koptok muvaffaqiyatli kiritilsa, bu protsedura `true`, agar U -barg allaqachon band bo'lgan bo'lsa, `false` qiymatini qaytaradi.
- Ushbu protseduraga ko'pi bilan 500 000 marotaba murojaat qilish mumkin.

Ikkinchi protsedura:

```
std::vector<int> collect()
```

- Barcha kiritilgan koptoklardagi qiymatlarni to'playdi, mashinani bo'shatadi va hosil bo'lgan S massivini qaytaradi.

Agar dastur ishlashi davomida istalgan vaqtda C qiymati 1000 dan oshsa, yechimingiz `Output isn't correct: Too many resources used` verdiktini oladi.

Mashinaning tuzilishi `find_structure` chaqirilishidan oldin aniqlangan bo'ladi. Grader **deterministik** bo'lib, agar siz uni ikki marta ishga tushirsangiz va ikkala ishga tushirish ham bir xil ketma-ketlikdagi amallarni bajarsa, `collect` ga qilingan chaqiruvlar bir xil massivlarni qaytaradi.

Constraints

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Subtasks

Subtask	Ball	Qo'shimcha Cheklovlar
1	5	Ildiz tugunning M ta bolasi mavjud
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Qo'shimcha cheklovlarsiz

3 va 4-subtasklarda, sizning ballingiz C ning qiymatiga quyidagicha bog'liq.

Subtask 3 (25 points)

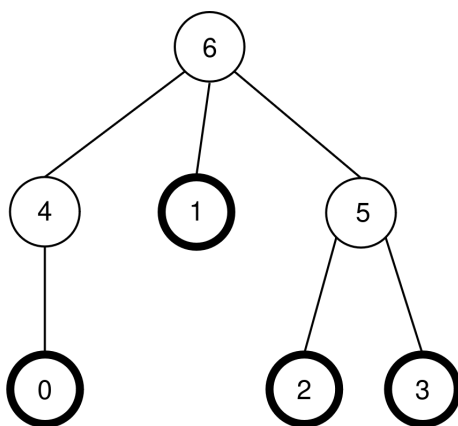
Limits	Ball
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subtask 4 (60 points)

Limits	Ball
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Example

Masala sharti berilishidagi mashinani ko'rib chiqaylik, ya'ni $N = 7$, $M = 4$ va $P = [4, 6, 5, 5, 6, 6]$. Mashina yana quyidagi rasmda ko'rsatilgan:



Grader quyidagi chaqiruv qiladi:

```
find_structure(4)
```

Protsedura quyidagi chaqiruvlar ketma-ketligini amalga oshirishi mumkin:

1. `insert(0, 0)` : 0-barg bo'sh, shuning uchun 0 qiymatiga ega koptok 0-bargga joylashtiriladi. $P[0] = 4$ tuguni bo'sh, shuning uchun koptok 4-tugunga o'tadi. $P[4] = 6$ tuguni bo'sh, shuning uchun koptok 6-tugunga o'tadi. 6-tugun ildiz bo'lgani uchun harakat to'xtaydi. Chaqiruv `true` qaytaradi.
2. `insert(3, 20)` : 3-barg bo'sh, shuning uchun 20 qiymatiga ega koptok 3-bargga joylashtiriladi. $P[3] = 5$ -tugun bo'sh, shuning uchun koptok 5-tugunga o'tadi. $P[5] = 6$ band bo'lganligi sababli, harakat to'xtaydi. Chaqiruv `true` qaytaradi.

3. `insert(1, 10)` : 1-barg bo'sh, shuning uchun 10 qiymatiga ega koptok 1-bargga joylashtiriladi. $P[1] = 6$ band bo'lgani uchun koptok 1 tugunida qoladi. Chaqiruv `true` qaytaradi.
4. `insert(2, 30)` : 2-barg bo'sh, shuning uchun 30 qiymatiga ega koptok 2-bargga joylashtiriladi. $P[2] = 5$ band bo'lgani uchun koptok 2-tugunda qoladi. Chaqiruv `true` qaytaradi.
5. `insert(1, 25)` : 1-barg band, shuning uchun koptok 1-bargga kiritilmaydi. Chaqiruv `false` qaytaradi.
6. `insert(0, 20)` : 0-barg bo'sh, shuning uchun 20 qiymatiga ega koptok 0-bargga joylashtiriladi. $P[0] = 4$ -tugun bo'sh, shuning uchun koptok 4-tugunga o'tadi. $P[4] = 6$ band bo'lganligi sababli, harakat to'xtaydi. Chaqiruv `true` qaytaradi.

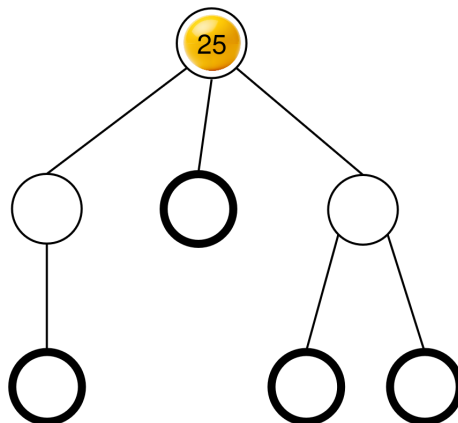
Olingan koptoklarning joylashuvi allaqachon masala sharti berilishida ko'rsatilgan edi.

Keyin, protsedura quyidagini chaqirishi mumkin:

```
collect()
```

Ushbu chaqiruvning natijasi masala tavsifida allaqachon tushuntirilgan edi, bu chaqiruv $S = [0, 10, 20, 30, 20]$ qaytaradi. Shundan so'ng, mashina yana bo'sh bo'ladi.

Keyin, protsedura `insert(2, 25)` ni chaqirishi mumkin. 2-barg bo'sh, shuning uchun 25 qiymatiga ega koptok 2-bargga joylashtiriladi. Keyin bu shar 5-tugunga, keyin esa 6-tugunga o'tadi va u yerda to'xtaydi. Chaqiruv `true` qaytaradi. Olingan koptoklarning joylashuvi quyidagi rasmda ko'rsatilgan.



Nihoyat, protsedura `collect()` ni chaqirishi mumkin, $S = [25]$ qaytadi va mashinani bo'shatiladi.

`find_structure` qaytara oladigan ikkita to'g'ri ota massivi mavjud: $R = P = [4, 6, 5, 5, 6, 6]$ (bu $L = [0, 1, 2, 3, 4, 5, 6]$ yorlig'iga mos keladi) va $R = [5, 6, 4, 4, 6, 6]$ (bu $L = [0, 1, 2, 3, 5, 4, 6]$ yorlig'iga mos keladi). Ushbu misolda, $K = 2$ va $B = 30$, shuning uchun $C = 32$.

Sample Grader

Input format:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Output format:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Bu yerda Q - `find_structure` tomonidan qaytarilgan R massivining uzunligi.